

Detektor spamu / phishingu SMS

Dokumentacja projektu

Autor	Julia Radosz, Małgorzata Pytlak
Przedmiot	Programowanie w języku Python — Projekt 3
Temat	Uczenie maszynowe — klasyfikacja binarna tekstu
Język / biblioteki	Python 3.10+, scikit-learn, Flask
Zbiór danych	SMS Spam Collection (UCI, CC BY 4.0)
Rok akademicki	2025/26

1. Opis projektu

Projekt rozwiązuje problem klasyfikacji binarnej: rozpoznawanie, czy wiadomość SMS jest treścią pożądaną (ham), czy spamem / próbą phishingu (spam). Zbiór jest niezbalansowany — spam stanowi ok. 13% danych — dlatego modele oceniane są przede wszystkim metryką F1 dla klasy spam, a nie samą dokładnością (trywialny klasyfikator „zawsze ham” miałby ~87% trafności, będąc bezużytecznym).

1.1 Problem i motywacja

Spam i phishing SMS to realne zagrożenie: wyłudzenie danych, oszustwa „na kod BLIK”, fałszywe dopłaty i linki phishingowe. Zadanie polega na automatycznym oznaczaniu takich wiadomości. Kluczową trudnością jest niezbalansowanie danych — klasyfikator musi skutecznie wychwytywać rzadką klasę (spam), a nie tylko maksymalizować ogólną trafność.

2. Uruchomienie

2.1 Wymagania

- Python 3.10 lub nowszy (testowano na 3.14),
- pip,
- zależności z requirements.txt: numpy, pandas, scipy, scikit-learn, matplotlib, wordcloud, joblib, flask.

2.2 Kroki

Zalecane wirtualne środowisko:

```
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate  # Linux / macOS
```

Instalacja zależności:

```
pip install -r requirements.txt
```

Pełny eksperyment (pobranie danych, trening, wyniki):

```
python main.py
```

Klasyfikacja pojedynczej wiadomości z wiersza poleceń:

```
python main.py --classify "WINNER!! Call now to claim"
```

Strona internetowa (interaktywny klasyfikator):

```
python app.py
# przeglądarka: http://127.0.0.1:5000
```

3. Struktura projektu

main.py	- punkt wejścia CLI (trening / klasyfikacja)
app.py	- serwer WWW Flask (strona z klasyfikatorem)
generate_docs.py	- generator tego dokumentu (PDF)
templates/	- szablon strony (HTML, Jinja2)
static/	- styl strony (CSS, motyw różowy)
data/	- zbiór danych (pobierany automatycznie)
results/	- wykresy, metryki, zapisany model
src/	
config.py	- ścieżki i parametry globalne
data_loader.py	- pobranie + wczytanie + obsługa braków
preprocessing.py	- czyszczenie tekstu + TF-IDF + podział
models.py	- definicje 7 modeli + siatki hiperparametrów
evaluation.py	- metryki + wykresy diagnostyczne

4. Zbiór danych i pochodzenie przykładów

Wykorzystano zbiór SMS Spam Collection z repozytorium UCI (5574 wiadomości, etykiety ham/spam, licencja CC BY 4.0). Jest pobierany automatycznie przy pierwszym uruchomieniu i zapisywany w data/SMSSpamCollection. Po usunięciu duplikatów i pustych wierszy do uczenia trafia ok. 5158 wiadomości.

4.1 Przykłady losowane na stronie

Przyciski spam / ham na stronie losują prawdziwą wiadomość danej klasy bezpośrednio z tego samego zbioru (funkcja `_random_example` w `app.py`) — każde kliknięcie zwraca inną wiadomość. Losowane przykłady przechodzą przez filtr treści wulgarnych / erotycznych (`_EXPLICIT_PATTERN`); filtr działa wyłącznie na podglądzie strony, natomiast zbiór treningowy i cytowany benchmark UCI pozostają nienaruszone (zachowanie reprodukowalności). Gdy zbioru nie da się wczytać, używany jest słownik awaryjny `FALLBACK_EXAMPLES`.

5. Przetwarzanie danych (pre-processing)

- obsługa braków: usunięcie pustych / niekompletnych i zduplikowanych wierszy,
- czyszczenie tekstu: małe litery, usunięcie znaków niealfabetycznych,
- maskowanie tokenami elementów typowych dla phishingu (URL, e-mail, numery premium, kwoty),
- wektoryzacja TF-IDF (unigramy + bigramy, usuwanie stop-words),
- redukcja wymiarowości: maks. 3000 cech (`max_features`) oraz odrzucenie tokenów bardzo rzadkich (`min_df`),
- podział 80/20 ze stratyfikacją (zachowanie proporcji klas).

6. Modele uczenia maszynowego

Zdefiniowano 7 modeli z różnych rodzin algorytmów; każdy jest strojony walidacją krzyżową (GridSearchCV, 5 podziałów, kryterium F1).

Model	Paradygmat	Strojony hiperparametr
Naïve Bayes	probabilistyczny	alpha
Regresja logistyczna	liniowy	C
k-NN	instancyjny	liczba sąsiadów k
Drzewo decyzyjne	regulowy	max_depth
Las losowy	zespół (bagging)	n_estimators, max_depth
SVM (jądro liniowe)	marginesowy	C
Sieć neuronowa (MLP)	sieć neuronowa	alpha, warstwy

7. Metryki i ocena

Liczone są wymagane metryki: dokładność (accuracy), logarytmiczna funkcja straty (log loss), macierz błędów oraz krzywe uczenia się. Dodatkowo — ze względu na niezbalansowanie — precyzja, czułość, F1 i pole pod krzywą ROC (ROC AUC). Dla sieci neuronowej zapisywana jest też krzywa funkcji straty w kolejnych epokach. Wszystkie wykresy trafiają do katalogu results/.

8. Wyniki

Model	Acc.	Log loss	Precyzja	Czułość	F1 (spam)	ROC AUC
Regresja logistyczna	0.9855	0.0589	0.9380	0.9453	0.9416	0.9974

SVM (jądro liniowe)	0.9855	0.0524	0.9380	0.9453	0.9416	0.9971
Sieć neuronowa (MLP)	0.9855	0.0601	0.9748	0.9062	0.9393	0.9914
Naive Bayes	0.9855	0.0562	0.9829	0.8984	0.9388	0.9893
Las losowy	0.9855	0.1636	0.9913	0.8906	0.9383	0.9984
k-NN	0.9777	0.4205	0.9817	0.8359	0.9030	0.9529
Drzewo decyzyjne	0.9641	0.4619	0.8321	0.8906	0.8604	0.9151

Najlepszy model: Regresja logistyczna — najwyższe $F1 = 0.9416$ (precyzja 0.9380, czułość 0.9453) przy ROC AUC = 0.9974 i dokładności = 0.9855. Wybór oparto na $F1$ dla klasy spam, ponieważ zbiór jest niezbilansowany (sama dokładność byłaby myląca). Przy zbliżonej skuteczności do SVM i sieci neuronowej regresja logistyczna jest modelem prostym, szybkim i interpretowalnym (wagi cech), co czyni ją wyborem najrozsądniejszym. Model wraz z wektoryzatorem zapisywany jest w `results/best_model.joblib` i używany przez stronę WWW oraz tryb `--classify`.

Dokładne wartości mogą się nieznacznie różnić zależnie od wersji bibliotek; powtarzalność zapewnia ustalone ziarno `RANDOM_STATE = 42`. Wszystkie wykresy (chmury słów, macierze błędów, krzywe uczenia, ROC, porównanie modeli) zapisują się w katalogu `results/`.

9. Strona internetowa

- lekki serwer Flask (`app.py`) wczytujący zapisany najlepszy model,
- własny szablon HTML (`templates/`) i własny CSS (`static/`) — motyw różowy z dbałością o kontrast (WCAG AA),
- formularz z werdyktem (spam / bezpieczna) i paskiem prawdopodobieństwa, legenda pojęć ham/spam,
- przyciski losujące prawdziwe przykłady ze zbioru (patrz sekcja 4.1) z filtrem treści nie stosownych,
- obsługa sytuacji bez wytrenowanego modelu (komunikat dla użytkownika).

9.1 Przykładowe wiadomości do testów

Gotowe wiadomości do ręcznego wklejenia w formularz na prezentacji (model trenowano na zbiorze anglojęzycznym, więc najlepiej testować po angielsku):

Spam / phishing:

WINNER!! You have won a 1000 cash prize. Call 09061701461 now to claim your reward!
URGENT! You have won a 1 week FREE membership in our 100,000 Prize Jackpot! Txt CLAIM to 81010

Ham (treść pożądana):

Hey, are we still meeting for lunch tomorrow at noon?
Thanks for dinner last night, see you on Sunday at grandma's place

10. Wkład AI i podział pracy

AI (Claude, Anthropic) wykorzystano wyłącznie do zadań pomocniczych, niewpływających na logikę uczenia maszynowego ani na metryki i wyniki:

- pula słów do filtrowania treści wulgarnych / erotycznych w przykładach losowanych na stronie (`_EXPLICIT_PATTERN` w `app.py`) — działa tylko na podglądzie, zbiór i modele bez zmian,
- stylizacja strony: `static/style.css` (motyw różowy, kontrast, responsywność) i drobne elementy prezentacyjne szablonu,
- skrypt generujący oraz wygenerowanie tego pliku PDF — układ, tabele, formatowanie (treść merytoryczna pochodzi od autorek),
- redakcja i formatowanie tekstu README oraz tej dokumentacji.

Cała logika eksperymentu ML oraz logika strony zostały opracowane przez autorki. Pracę podzielono na dwa

równe filary:

10.1 Julia Radosz — dane, pre-processing i modele

- `config.py`, `data_loader.py` — parametry, pobranie danych, obsługa braków i duplikatów,
- `preprocessing.py` — czyszczenie tekstu, maskowanie tokenami, TF-IDF, podział ze stratyfikacją,
- `models.py` — dobór i strojenie 7 modeli (siatki hiperparametrów),
- `pipeline.py` (część) — trening, `GridSearchCV`, wybór najlepszego modelu; `main.py` — tryb CLI.

10.2 Małgorzata Pytlak — ewaluacja, wizualizacje, strona

- `evaluation.py` — accuracy, log loss, macierz błędów, krzywe uczenia, F1, ROC AUC,
- `visualization.py` — chmury słów, rozkłady, porównanie modeli,
- `pipeline.py` (część) — analiza eksploracyjna, zapis wyników,
- `app.py` + `templates/index.html` — strona: klasyfikacja i losowanie przykładów; treść merytoryczna dokumentacji.

Każda z autorek zna i potrafi omówić swój obszar podczas dyskusji. Niniejszy dokument jest jedynym, kompletnym źródłem informacji o wkładzie własnym, użyciu AI oraz bibliografii (sekcja 12).

10.3 Elementy z samouczków / dokumentacji

API i dobór parametrów modeli opierają się na oficjalnej dokumentacji `scikit-learn` (zob. sekcja 12.3). Kod nie kopiuje gotowych samouczków 1:1 — wzorce użycia (`GridSearchCV`, `TfidfVectorizer`, `learning_curve`) pochodzą z dokumentacji biblioteki i zostały dostosowane do specyfiki zadania.

11. Znane ograniczenia

- model trenowany na zbiorze anglojęzycznym — wiadomości po polsku klasyfikuje słabo (słownik cech jest angielski),
- filtr treści niestosownych jest oparty na liście słów — może pominąć nietypowe sformułowania,
- brak modeli głębokich (np. transformerów) — świadomy wybór na rzecz przejrzystości i czasu uczenia,
- wyniki mogą się minimalnie różnić zależnie od wersji bibliotek (powtarzalność zapewnia ustalone ziarno losowości).

12. Bibliografia i źródła

Sekcja stanowi pełną bibliografię projektu. Brak lub błędne cytowanie źródeł wpływa na ocenę — lista poniżej jest kompletna i obejmuje zbiór danych, biblioteki (z wersjami), źródła internetowe (z datą dostępu) oraz wykorzystane narzędzie AI.

12.1 Zbiór danych

- Almeida, T. & Hidalgo, J. (2011). SMS Spam Collection [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5CC84> — <https://archive.ics.uci.edu/dataset/228/sms+spam+collection> (data dostępu: 2026-05-19). Licencja: CC BY 4.0.

12.2 Biblioteki i frameworki (nazwa + wersja)

- NumPy 2.4.1 — <https://numpy.org>
- pandas 3.0.3 — <https://pandas.pydata.org>
- SciPy 1.17.1 — <https://scipy.org>
- scikit-learn 1.8.0 — <https://scikit-learn.org>
- Matplotlib 3.10.8 — <https://matplotlib.org>
- WordCloud 1.9.6 — https://github.com/amueller/word_cloud
- joblib 1.5.3 — <https://joblib.readthedocs.io>
- Flask 3.1.2 — <https://flask.palletsprojects.com>

- fpdf2 2.8.7 — <https://py-pdf.github.io/fpdf2/> (generator tej dokumentacji).

12.3 Dokumentacja / źródła internetowe

- [scikit-learn — User Guide](https://scikit-learn.org/stable/user_guide.html) (klasyfikacja, dobór modelu, metryki).
https://scikit-learn.org/stable/user_guide.html (data dostępu: 2026-05-19).

12.4 Narzędzia AI

- Model: Claude Opus 4.7 (claude-opus-4-7).
- Producent: Anthropic.
- Zastosowanie: zadania pomocnicze, niemerytoryczne — filtr treści na stronie, stylizacja (CSS), skrypt i formatowanie tej dokumentacji PDF oraz redakcja tekstu (zob. sekcja 10).
- Data dostępu: 2026-05-19.
- Zakres weryfikacji: projekt uruchomiono i przetestowano (pełny pipeline, klasyfikacja, strona WWW); każda z autorek weryfikuje i rozumie swój obszar pracy przed prezentacją.